QUESTION: 3 GROUP: **Coding** SECTION: Generic Coding 1 Mark(s): 10

You are given a directed graph represented by a list of edges, where each edge is a tuple of two vertices (u, v), indicating an edge from vertex u to vertex v. The vertices listed in the input are the only vertices that exist in the graph.

Your task is to write a program that calculates the in-degree of each vertex in the graph and print the vertex name and its indegree of the vertex that has the highest in-degree. if more than one node has the highest in-degree then print the required details of the vertex that appears first when the vertices are arranged in alphabetically ascending order.

The in-degree of a vertex in a directed graph is the count of incoming edges to that vertex, i.e., the number of edges that have the vertex as the destination.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

Constraints:

i. The input list edges will have at most 10^4 edges.
ii. The vertices in the input list edges are unique and can be represented as strings.
iii. The graph can have self-loops, i.e., edges of the form (u, u), where u is a vertex. Such self-loops should be counted as incoming edges.

Input Format:

The first line of input should consist of *N*, the number of edges.
The next *N* line of input should consist of *N* edges, where each edge contains two vertices (u, v) representing an edge from vertex u to vertex v, both separated by a single white space. The vertices are represented as strings

Output Format:

The output should print the highest node having indegree where th
is the vertices of the graph and the second column is indegree, bot
by a single white space. If two or more nodes have the same highe

inapp.hirepro.ai is sh

Previous

1080

The nodes with the highest in-degree are C and D, with an in-degree of 2 but
Vertex C appears first when they are arranged alphabetically in ascending order
so we will consider C as output.
Therefore, the output is "C 2".

Sample Input 2:

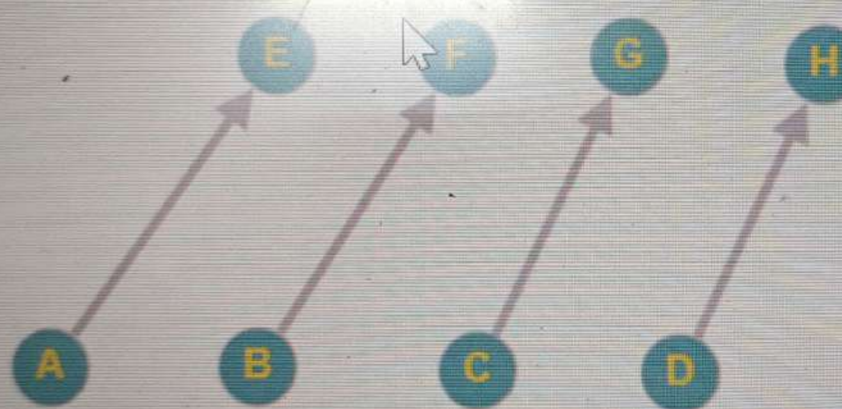
4
A E
B F
C G
D H

Sample Output 2:

E 1

Explanation 2:

After analyzing the edges, we can calculate the indegree of each vertex as follows:



Nodes with the highest in-degree are E, F, G, and H, with an in-degree of 1. But, here E appears first when they are arranged alphabetically in ascending order, so we will consider E as an output.
Therefore, the output is "E 1".

The nodes with the highest in-degree are C and D, with an in-degree of 2 but Vertex C appears first when they are arranged alphabetically in ascending order so we will consider C as output.
Therefore, the output is "C 2".

Sample Input 2:

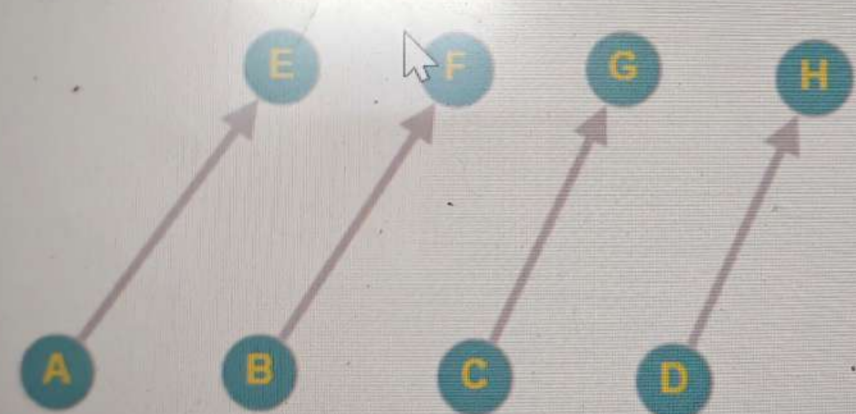
4
A E
B F
C G
D H

Sample Output 2:

E 1

Explanation 2:

After analyzing the edges, we can calculate the indegree of each vertex as follows:



Nodes with the highest in-degree are E, F, G, and H, with an in-degree of 1. But, here E appears first when they are arranged alphabetically in ascending order, so we will consider E as an output.
Therefore, the output is "E 1".

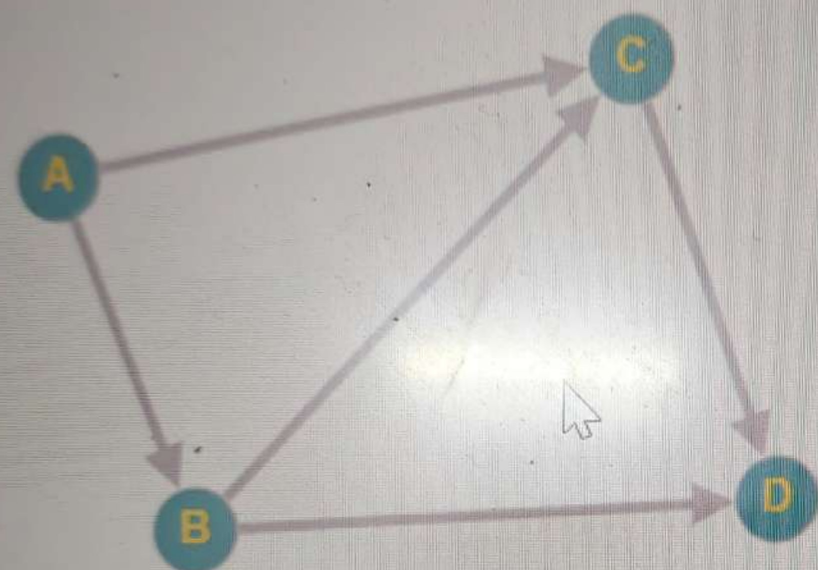
C D
B D
A C

Sample Output 1:

C 2

Explanation 1:

After analyzing the edges, we can calculate the indegree of each vertex as follows:



Vertex A has an in-degree of 0 because it does not have any incoming edges.
Vertex B has an in-degree of 1 because one edge has B as the destination vertex: (A, B).
Vertex C has an in-degree of 2 because two edges have C as the destination vertex: (A, C) and (B, C).
Vertex D has an in-degree of 2 because two edges have D as the destination vertex: (C, D) and (B, D).
The nodes with the highest in-degree are C and D, with an in-degree of 2 but Vertex C appears first when they are arranged alphabetically in asc
so we will consider C as output.

arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

Constraints:

i. The input list edges will have at most 10^4 edges.
ii. The vertices in the input list edges are unique and can be represented as strings.
iii. The graph can have self-loops, i.e., edges of the form (u, u), where u is a vertex. Such self-loops should be counted as incoming edges.

Input Format:

The first line of input should consist of *N*, the number of edges.
The next *N* line of input should consist of *N* edges, where each edge contains two vertices (u, v) representing an edge from vertex u to vertex v, both separated by a single white space. The vertices are represented as strings

Output Format:

The output should print the highest node having indegree where the first column is the vertices of the graph and the second column is indegree, both separated by a single white space. If two or more nodes have the same highest indegree, then pick the vertex that appears first when they are arranged in alphabetically ascending order.

Sample Input 1:

```
5
A B
B C
C D
B D
A C
```

Sample Output 1:

```
C 2
```


Previous

080

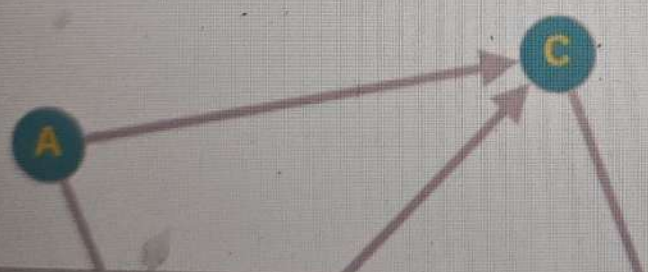
Input Format:
The first line of input should consist of N, the number of edges.
The next N line of input should consist of N edges, where each edge contains two vertices (u, v) representing an edge from vertex u to vertex v, both separated by a single white space. The vertices are represented as strings

Output Format:
The output should print the highest node having indegree where the first column is the vertices of the graph and the second column is indegree, both separated by a single white space. If two or more nodes have the same highest indegree, then pick the vertex that appears first when they are arranged in alphabetically ascending order.

Sample Input 1:
5
A B
B C
C D
B D
A C

Sample Output 1:
C 2

Explanation 1:
After analyzing the edges, we can calculate the indegree of each vertex as follows:



(2,6)->(2,7). Then next move is "R 6" so the snake turns right, and moves 6 units:
(2,7)->(3,7)->(4,7)->(5,7)->(6,7)->(7,7)->(8,7). The next move is "R 2" so the snake turns right again and moves 2 units: (8,7)->(8,6)->(8,5)
Since the snake did not hit the poison or fall off the grid, it is alive at the end of the game. The snake travelled 4+6+2 = 12 units after which the game ended, by which time the snake had reached the final position (8,5). Hence the output is printed as
12
(8,5) A

Sample Input2:

4 5
3 2
4 L 2 L 1 R 2 L 3

Sample Output2:

5
(2,6) D

Explanation2:

From the input line 1, the grid is of width 4 (x-axis) and height 5 (y-axis).
The snake starts at (3, 2) and is facing right i.e. looking towards (4, 2). The first move is "L 2" so the snake turns left and moves 2 units i.e. (3,2)->(3,3)->(3,4).
Then next move is "L 1" so the snake turns left again and moves 1 unit i.e. (3,4)->(2,4). The next move is "R 2" so the snake turns right and moves 2 units i.e. (2,4)->(2,5)->(2,6). Given the grid is of height 5 only, when trying to move to (2,6) the snake falls off the grid and dies. The game hence ends here, even though there was one move left for the snake.
The snake travelled for 2+1+2 = 5 units, and the snake was at (2,6) when the game ended. Hence the output is printed as
5
(2,6) D

3 L 4 R 6 R 2

Sample Output1:

12
(8,5) A

Explanation1:

From the input line 1, the grid is of width 10 (x-axis) and height 8 (y-axis).
The snake starts at (2, 3) and is facing right i.e. looking towards (3, 3). The first move is "L 4" so the snake turns left and moves 4 units i.e. (2,3)->(2,4)->(2,5)->(2,6)->(2,7). Then next move is "R 6" so the snake turns right, and moves 6 units: (2,7)->(3,7)->(4,7)->(5,7)->(6,7)->(7,7)->(8,7). The next move is "R 2" so the snake turns right again and moves 2 units: (8,7)->(8,6)->(8,5)
Since the snake did not hit the poison or fall off the grid, it is alive at the end of the game. The snake travelled 4+6+2 = 12 units after which the game ended, by which time the snake had reached the final position (8,5). Hence the output is printed as
12
(8,5) A

Sample Input2:

4 5
3 2
4 L 2 L 1 R 2 L 3

Sample Output2:

5
(2,6) D

Explanation2:

From the input line 1, the grid is of width 4 (x-axis) and height 5 (y-a
The snake starts at (3, 2) and is facing right i.e. looking towards (4,

Constraints:

$1 <= W, D <= 100$
$1 <= x <= W$
$1 <= y <= D$
$1 <= len_i <= 10$
$dir_i = \{S,L,R\}$
$n > 0$

Sample Input1:

10 8
2 3
3 L 4 R 6 R 2

Sample Output1:

12
(8,5) A

Explanation1:

From the input line 1, the grid is of width 10 (x-axis) and height 8 (y-axis).
The snake starts at (2, 3) and is facing right i.e. looking towards (3, 3). The first move is "L 4" so the snake turns left and moves 4 units i.e. (2,3)->(2,4)->(2,5)->(2,6)->(2,7). Then next move is "R 6" so the snake turns right, and moves 6 units: (2,7)->(3,7)->(4,7)->(5,7)->(6,7)->(7,7)->(8,7). The next move is "R 2" so the snake turns right again and moves 2 units: (8,7)->(8,6)->(8,5)
Since the snake did not hit the poison or fall off the grid, it is alive at the end of the game. The snake travelled 4+6+2 = 12 units after which the game ended, by which time the snake had reached the final position (8,5). Hence the output is printed as
12
(8,5) A

Sample Input2:

Previous

080

arbitrary strings anywhere in the program, as these contribute to the output and test cases will fail.

Input Format:

The first line contains 2 integers W and D denoting the dimensions of the grid.
The second line contains 2 integers x and y denoting the starting position of the snake.
The third line is of the form n dir_1 len_1 dir_2 len_2 ... dir_n len_n where n denotes the number of moves the snake makes; dir_1 denotes the direction for 1st move and similarly len_1 denotes the length of the 1st move, and so on.

Output Format:
The first line of output prints an integer denoting the total distance the snake travels until the game ends.
The second line of output prints the end co-ordinates for the snake in form (x2,y2) and status in form D for Dead or A for Alive, with a single space in between

Constraints:
$1 <= W, D <= 100$
$1 <= x <= W$
$1 <= y <= D$
$1 <= len_i <= 10$
$dir_i = \{S,L,R\}$
$n > 0$

Sample Input1:
10 8
2 3
3 L 4 R 6 R 2

Sample Output1:

Flag this question video game, a snake moves inside a two-dimensional square grid based on a set of fixed instructions. As the snake traverses its path in the grid, it leaves behind a trail of poison along the path it travels. If the snake crosses that path again, the poison will kill it, and thus end the game. A second way to end the game is if the snake falls off the grid.

The square grid is a representation of the first quadrant of the cartesian plane i.e. (0, 0) is at bottom left of the grid, x values increase as we go right, and y values increase as we go up.

Given the grid, starting position and the path of the snake during the game, write a program to determine the final position of the snake when the game ends, and if the snake is alive or dead at the end.

The starting coordinates of the snake are given as (x, y). Initially, the snake faces to the east, that is, in the same direction as the positive x-axis. The path is described as a series of moves in the form $[T, k]$. T indicates the direction turned, which is either 'S', 'L', or 'R', meaning Straight, Left, or Right, respectively, while k is the distance to travel after turning. Thus "R 6" means to turn right (relative to the current direction) and then travel 6 units.

Read the input from STDIN and print the output to STDOUT. Do not print arbitrary strings anywhere in the program, as these contribute to the output and test cases will fail.

Input Format:

The first line contains 2 integers W and D denoting the dimensions of the grid.
The second line contains 2 integers x and y denoting the starting position of the snake.
The third line is of the form n dir_1 len_1 dir_2 len_2 ... dir_n len_n where n denotes the number of moves the snake makes; dir_1 denotes the direction for 1st move and similarly len_1 denotes the length of the 1st move, and so on.

Output Format:

Flag this question video game, a snake moves ... based on a set of fixed instructions. As the snake traverses its path in the grid, it leaves behind a trail of poison along the path it travels. If the snake crosses that path again, the poison will kill it, and thus end the game. A second way to end the game is if the snake falls off the grid.

The square grid is a representation of the first quadrant of the cartesian plane i.e. (0, 0) is at bottom left of the grid, x values increase as we go right, and y values increase as we go up.

Given the grid, starting position and the path of the snake during the game, write a program to determine the final position of the snake when the game ends, and if the snake is alive or dead at the end.

The starting coordinates of the snake are given as (x, y). Initially, the snake faces to the east, that is, in the same direction as the positive x-axis. The path is described as a series of moves in the form $[T, k]$. T indicates the direction turned, which is either 'S', 'L', or 'R', meaning Straight, Left, or Right, respectively, while k is the distance to travel after turning. Thus "R 6" means to turn right (relative to the current direction) and then travel 6 units.

Read the input from STDIN and print the output to STDOUT. Do not print arbitrary strings anywhere in the program, as these contribute to the output and test cases will fail.

Input Format:

The first line contains 2 integers W and D denoting the dimensions of the grid.
The second line contains 2 integers x and y denoting the starting position of the snake.
The third line is of the form n dir_1 len_1 dir_2 len_2 ... dir_n len_n where n denotes the number of moves the snake makes; dir_1 denotes the direction for 1st move and similarly len_1 denotes the length of the 1st move, and so on.

Output Format:

inapp.hirepro.ai is s

Previous

080

4 2 0
3 0 1 2

Explanation:
Given Data :

	R1	R2	R3	R4
T1	P1	P3	P2	P4
T2	P4	P2	P3	P1
T3	P1	P3	P2	P4

Player 1 vs Player 2: T1 is selected as winning and losing ranks are same for T1 and T3, hence order of priority T1>T2>T3.
Player 1 vs Player 3: T1 is selected as winning and losing ranks are same for T1 and T3, hence order of priority T1>T2>T3.
Player 1 vs Player 4: T1 is selected as winning and losing ranks are same for T1, T2 and T3, hence order of priority T1>T2>T3.
Player 2 vs Player 3: T1 is selected as winning and losing ranks are same for T1, T2 and T3, hence order of priority T1>T2>T3.
Player 2 vs Player 4: T2 is selected as winning rank (1) is better than for T1 (3) and T3 (3).
Player 3 vs Player 4: T2 is selected as winning rank (1) is better tha
and T3 (2).

inapp.hirepro.ai

Previo

1080

Sample Input 2:

4
1 3 2 4
4 2 3 1
1 3 2 4

Sample Output 2:

4 2 0
3 0 1 2

Explanation:

Given Data :

	R1	R2	R3	R4
T1	P1	P3	P2	P4
T2	P4	P2	P3	P1
T3	P1	P3	P2	P4

Player 1 vs Player 2: T1 is selected as winning and losing ranks are same for T1 and T3, hence order of priority T1>T2>T3.
Player 1 vs Player 3: T1 is selected as winning and losing ranks are same for T1 and T3, hence order of priority T1>T2>T3.
Player 1 vs Player 4: T1 is selected as winning and losing ranks are T1, T2 and T3, hence order of priority T1>T2>T3.

Player 2 vs Player 3: P2 beats P3 in T1 & T3, while P3 beats P2 in T2. Rank of winner P2 in T1 is 2, while rank of winner P3 in T2 is 1 and winner P2 rank in T3 is 1. Hence T1 is eliminated. Since ranks of winners in T2 and T3 are same, as second step we look at best rank of losers. In T2 loser P2 has rank 2 while in T3 loser P3 also has rank 2. Since loser ranks are also same, as step three the priority is given as T1>T2>T3. Hence T2 is chosen for this race and P3 is the winner.

Summing up, T1 is chosen for 2 races, T2 for 1 and T3 for nil. Hence first line of output is 2 1 0.
P1 wins twice, P2 does not win any race, and P3 wins 1 race. Hence second line of output is 2 0 1.

Sample Input 2:

4
1 3 2 4
4 2 3 1
1 3 2 4

Sample Output 2:

4 2 0
3 0 1 2

Explanation:

Given Data :

	R1	R2	R3	R4
T1	P1	P3	P2	P4

T3	P2 P3 P1

Player 1 vs Player 2: P1 beats P2 in T1, while P2 beats P1 in T2 and T3. Rank of winner P1 in T1 is 1, while rank of winner P2 in T2 is 2 and winner P2 rank in T3 is 1. Hence T2 is eliminated. Since ranks of winners in T1 and T3 are same, as second step we look at best rank of losers. In T1 loser P2 has rank 2 while in T3 loser P1 has rank 3. Since rank 2 is better for loser, T1 is chosen for this race and P1 is the winner.

Player 1 vs Player 3: P1 beats P3 in T1, while P3 beats P1 in T2 and T3. Rank of winner P1 in T1 is 1, while rank of winner P3 in T2 is 1 and winner P3 rank in T3 is 2. Hence T3 is eliminated. Since ranks of winners in T1 and T2 are same, as second step we look at best rank of losers. In T1 loser P3 has rank 3 while in T2 loser P1 also has rank 3. Since loser ranks are also same, as step three the priority is given as T1>T2>T3. Hence T1 is chosen for this race and P1 is the winner.

Player 2 vs Player 3: P2 beats P3 in T1 & T3, while P3 beats P2 in T2. Rank of winner P2 in T1 is 2, while rank of winner P3 in T2 is 1 and winner P2 rank in T3 is 1. Hence T1 is eliminated. Since ranks of winners in T2 and T3 are same, as second step we look at best rank of losers. In T2 loser P2 has rank 2 while in T3 loser P3 also has rank 2. Since loser ranks are also same, as step three the priority is given as T1>T2>T3. Hence T2 is chosen for this race and P3 is the winner.

Summing up, T1 is chosen for 2 races, T2 for 1 and T3 for nil. Hence first line of output is 2 1 0.
P1 wins twice, P2 does not win any race, and P3 wins 1 race. Henc

2 1 0
2 0 1

Explanation:
There are three racers.
In Hilly terrain, Player 1 is Rank 1, Player 2 is Rank 2 and Player 3 is Rank 3.
In City Race, Player 3 is Rank 1, Player 2 is Rank 2 and Player 1 is Rank 3.
In Off Roading, Player 2 is Rank 1, Player 3 is Rank 2 and Player 1 is Rank 3.

	R1	R2	R3
T1	P1	P2	P3
T2	P3	P2	P1
T3	P2	P3	P1

Player 1 vs Player 2: P1 beats P2 in T1, while P2 beats P1 in T2 and T3. Rank of winner P1 in T1 is 1, while rank of winner P2 in T2 is 2 and winner P2 rank in T3 is 1. Hence T2 is eliminated. Since ranks of winners in T1 and T3 are same, as second step we look at best rank of losers. In T1 loser P2 has rank 2 while in T3 loser P1 has rank 3. Since rank 2 is better for loser, T1 is chosen for this race and P1 is the winner.

Player 1 vs Player 3: P1 beats P3 in T1, while P3 beats P1 in T2 an
winner P1 in T1 is 1, while rank of winner P3 in T2 is 1 and winner F

inapp.hirepro.ai is

Previou

080

racer (right most) is rank N (weakest).

Output Format:
The first line of output contains three integers that determine the number of races that will happen in each terrain (Hilly, City, Off-road).
The second line of output contains N space-separated integers that are the wins of the individual players from 1 to N.

Sample Input 1:
3
1 2 3
3 2 1
2 3 1

Sample Output 1:
2 1 0
2 0 1

Explanation:
There are three racers.
In Hilly terrain, Player 1 is Rank 1, Player 2 is Rank 2 and Player 3 is Rank 3.
In City Race, Player 3 is Rank 1, Player 2 is Rank 2 and Player 1 is Rank 3.
In Off Roading, Player 2 is Rank 1, Player 3 is Rank 2 and Player 1 is Rank 3.

	R1	R2	R3
T1	P1	P2	P3
T2	P3	P2	P1

3. If the winning & losing ranks in terrains are still equal, then the terrains are chosen in this order : Hilly terrain first, City Race second and Off-Roading last.

You're provided the rank list of each racer on the three terrains. Write a program to predict how many races will happen on each terrain, and the wins of each racer.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

Input Format:

The First line of input contains N, the number of racers. The racers are numbered 1 to N.
The second line of input contains N space-separated integers (1 to N) that determines the rank list of Hilly terrain.
The third line of input contains N space-separated integers (1 to N) that determines the rank list of City Race.
The fourth line of input contains N space-separated integers (1 to N) that determines the rank list of Off-Roading.
In all the rank lists, the first racer (left most) is rank 1 (strongest) and the last racer (right most) is rank N (weakest).

Output Format:

The first line of output contains three integers that determine the number of races that will happen in each terrain (Hilly, City, Off-road).
The second line of output contains N space-separated integers that are the wins of the individual players from 1 to N.

Sample Input 1:

3
1 2 3

inapp.hirepro.ai is

Previous

080

A racing tournament is happening, and racers from various backgrounds have come to participate. There are three racing terrains:

1. Hilly terrain
2. City Race
3. Off-Roading

For each terrain, the racers are ranked based on their expertise. The strongest racer has rank 1. A racer with a lower rank always beats a racer with a higher rank, on that terrain.

The tournament is in round-robin format i.e. each contestant races once against all other contestants, one on one. Since two racers compete only once, the choice of terrain becomes very important. To make the tournament exciting, the terrain for each race is selected by the organizers based on the ranking lists, as follows.

1. Each race should be raced on the terrain in which the winner would be the strongest i.e. winner must have the lowest rank in the corresponding rank list.
2. In case of similar ranking in step 1 e.g. racer 1 wins on Hilly terrain and racer 2 wins in City Race, and they are both ranked the same on the corresponding rank lists, then the organizers would choose the terrain for which the loser would have the best rank.
3. If the winning & losing ranks in terrains are still equal, then the terrains are chosen in this order : Hilly terrain first, City Race second and Off-Roading last.

You're provided the rank list of each racer on the three terrains. Write a program to predict how many races will happen on each terrain, and the wins of each racer.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

Input Format:

The First line of input contains N, the number of racers. The racers

Previous

080

A racing tournament is happening, and racers from various backgrounds have come to participate. There are three racing terrains:

1. Hilly terrain
2. City Race
3. Off-Roading

For each terrain, the racers are ranked based on their expertise. The strongest racer has rank 1. A racer with a lower rank always beats a racer with a higher rank, on that terrain.

The tournament is in round-robin format i.e. each contestant races once against all other contestants, one on one. Since two racers compete only once, the choice of terrain becomes very important. To make the tournament exciting, the terrain for each race is selected by the organizers based on the ranking lists, as follows.

1. Each race should be raced on the terrain in which the winner would be the strongest i.e. winner must have the lowest rank in the corresponding rank list.
2. In case of similar ranking in step 1 e.g. racer 1 wins on Hilly terrain and racer 2 wins in City Race, and they are both ranked the same on the corresponding rank lists, then the organizers would choose the terrain for which the loser would have the best rank.
3. If the winning & losing ranks in terrains are still equal, then the terrains are chosen in this order : Hilly terrain first, City Race second and Off-Roading last.

You're provided the rank list of each racer on the three terrains. Write a program to predict how many races will happen on each terrain, and the wins of each racer.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

Input Format:

The First line of input contains N, the number of racers. The racers racer has rank 1. A racer with a lower rank always beats a racer with a higher rank, on that terrain.

The tournament is in round-robin format i.e. each contestant races once against all other contestants, one on one. Since two racers compete only once, the choice of terrain becomes very important. To make the tournament exciting, the terrain for each race is selected by the organizers based on the ranking lists, as follows.

1. Each race should be raced on the terrain in which the winner would be the strongest i.e. winner must have the lowest rank in the corresponding rank list.
2. In case of similar ranking in step 1 e.g. racer 1 wins on Hilly terrain and racer 2 wins in City Race, and they are both ranked the same on the corresponding rank lists, then the organizers would choose the terrain for which the loser would have the best rank.
3. If the winning & losing ranks in terrains are still equal, then the terrains are chosen in this order : Hilly terrain first, City Race second and Off-Roading last.

You're provided the rank list of each racer on the three terrains. Write a program to predict how many races will happen on each terrain, and the wins of each racer.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings anywhere in the program, as these contribute to the standard output and test cases will fail.

<u>Input Format:</u>
The First line of input contains N, the number of racers. The racers